

Probeklausur 1

Grundlagen der Programmierung

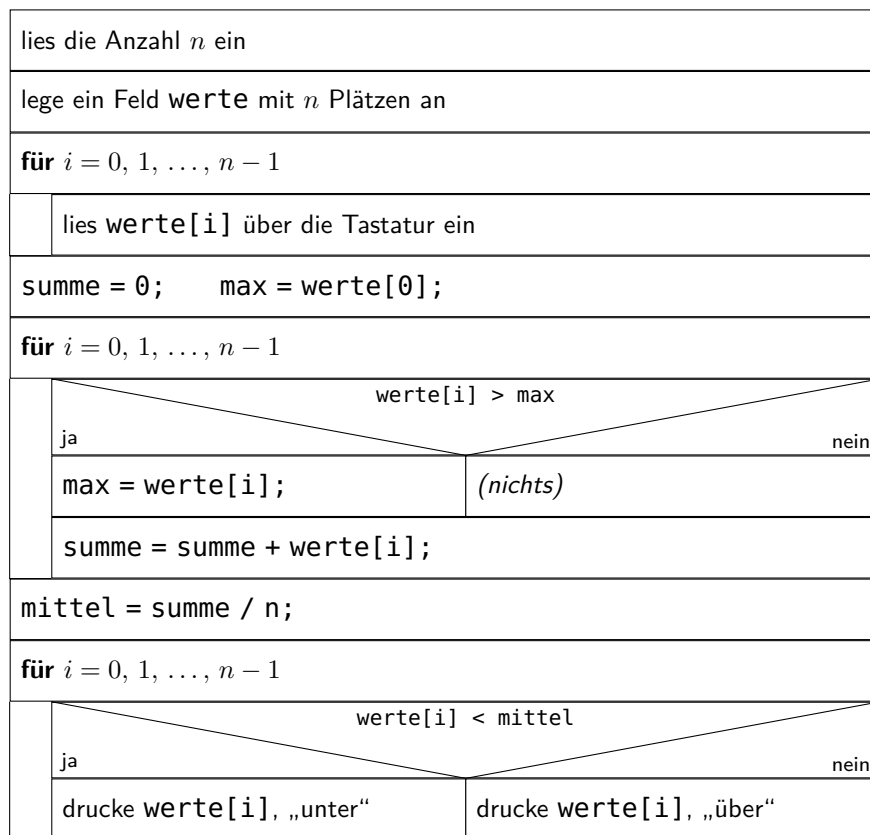
Studiengänge:	PEB & MVB	Autor:	Luis Staudt
Semester:	Sommersemester 2026	Schwierigkeit:	einfach
Bearbeitungszeit:	90 Minuten	Gesamtpunkte:	90

*Bearbeiten Sie alle fünf Aufgaben. Achten Sie auf sauberen, kompilierfähigen Java-Quelltext.
Hilfsmittel: OpenBook, keine elektronischen Geräte.*

Aufgabe 1: Struktogramm in Java umsetzen**(24 Punkte)**

Gegeben ist das folgende Struktogramm. Es liest eine Anzahl von Messwerten ein, bestimmt deren größten Wert und Mittelwert und gibt anschließend zu jedem Wert aus, ob er unter oder über dem Mittelwert liegt.

Setzen Sie das Struktogramm **vollständig in ein lauffähiges Java-Programm** um. Verwenden Sie zum Einlesen `Tastatureingabe leseInt()` bzw. `Tastatureingabe leseDouble()`.



Aufgabe 2: Bewegungsformel auswählen**(14 Punkte)**

Ein Körper bewegt sich geradlinig. Je nach Bewegungsart beschreibt eine andere Formel die zurückgelegte Strecke s .

Gegebene Formeln:

(1) gleichförmig: $s = v \cdot t$ (2) aus Ruhe beschleunigt: $s = \frac{1}{2} a t^2$

(3) mit Anfangsgeschwindigkeit: $s = v_0 \cdot t + \frac{1}{2} a t^2$

Schreiben Sie ein Programm, das den Benutzer zunächst die Bewegungsart per Zahl (1, 2 oder 3) wählen lässt und anschließend die benötigten Größen abfragt. Berechnen und drucken Sie die Strecke s mit der **richtigen** Formel. Fangen Sie eine **negative Zeit** ($t < 0$) als ungültige Eingabe ab. Die Auswahl ist mit `if/else` (kein `switch` nötig) zu realisieren.

Beispieldialog:

```
1 Bewegungsart (1=gleichfoermig, 2=beschleunigt, 3=mit v0): 2
2 Beschleunigung a [m/s^2]: 2.5
3 Zeit t [s]: 4
4 Zurueckgelegte Strecke s = 20.0 m
```

Aufgabe 3: Punkt im Kreis**(20 Punkte)**

Gegeben ist die aus Vorlesung und Übung bekannte Klasse `Punkt` (Attribute `xKoordinate`, `yKoordinate` als `double`).

Implementieren Sie die Methode `liegtInnerhalb`, die zurückgibt, ob die aufrufende `Punkt`-instanz innerhalb eines Kreises mit Radius r um einen übergebenen Mittelpunkt liegt. Übergebeparameter sind der Radius sowie der Mittelpunkt (ein `Punkt`).

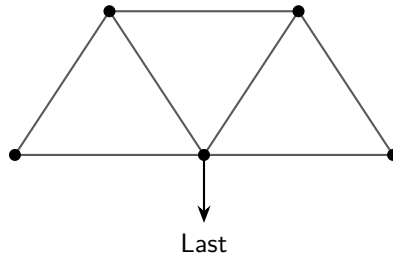
Rückgabe	Bedeutung
1	Punkt liegt innerhalb des Kreises oder auf dem Umfang
0	Punkt liegt außerhalb des Kreises
-1	ungültiger Kreis (Radius ≤ 0)

Wiederverwendung. Wer die aus der Übung bekannte Methode `berechneAbstandZuPunkt(Punkt p)` sinnvoll wiederverwendet, erhält einen **Zusatzpunkt**.

```
1 Punkt p1 = new Punkt();
2 p1.xKoordinate = 1.0; p1.yKoordinate = 1.5;
3 Punkt mp = new Punkt();
4 mp.xKoordinate = 5.6; mp.yKoordinate = -4.3;
5
6 int i = p1.liegtInnerhalb(3.0, mp);
7 if (i == 1)      System.out.println("p1 liegt innerhalb");
8 else if (i == 0) System.out.println("p1 liegt ausserhalb");
9 else            System.out.println("ungueltiger Radius");
```

Aufgabe 4: Fachwerk, Stäbe klassifizieren**(16 Punkte)**

Gegeben ist die aus Vorlesung und Übung bekannte Datei `fachwerk.jar`. Nach der Berechnung soll für ein gewähltes Fachwerk gezählt werden, wie viele **Zugstäbe**, **Druckstäbe** und **Nullstäbe** es enthält.



Schematische Darstellung eines Fachwerks.

Nullstab über Epsilon-Umgebung. Die Berechnung liefert für Nullstäbe sehr kleine Werte ($\neq 0$), z. B. `3.55E-15`. Erkennen Sie einen Nullstab über eine **Epsilon-Umgebung**:

$$|F| < \varepsilon \Rightarrow \text{Nullstab}, \quad F > \varepsilon \Rightarrow \text{Zugstab}, \quad F < -\varepsilon \Rightarrow \text{Druckstab}.$$

Benötigte Methoden: `int gibAnzahlStaebe()`, `Fachwerkstab gibStab(int i)`, `double gibStabkraft()`.

Ergänzen Sie den folgenden Quelltext, sodass die drei Anzahlen ausgegeben werden:

```

1 public class Fachwerkaufgabe {
2     public static void main(String[] args) {
3         System.out.println("Welches Fachwerk wollen Sie rechnen (0-4)?");
4         int wahl = Tastatureingabe.leseInt();
5         Fachwerk fw = new Fachwerk(wahl);
6         fw.berechneGroessen();
7         double epsilon = 1e-6;
8         // hier erweitern: zaehlen und ausgeben
9     }
10 }
```

Aufgabe 5: Wertetabelle in eine Datei schreiben**(16 Punkte)**

Ein Körper fällt aus der Ruhe; für die zurückgelegte Strecke gilt

$$s(t) = \frac{1}{2} a t^2 \quad \text{mit } a = 9,81 \text{ m/s}^2.$$

Schreiben Sie ein Programm, das für die Zeitpunkte $t = 0 \text{ s}$ bis $t = 15 \text{ s}$ in Schritten von 1 s jeweils die Strecke berechnet und Zeile für Zeile in die Datei `fall.txt` schreibt.

Hinweis. Verwenden Sie `java.io.PrintWriter`: Datei mit `new PrintWriter("fall.txt")` öffnen, je Zeile mit `write(...)` schreiben (Zeilenumbruch `"\n"` nicht vergessen) und am Ende `close()` aufrufen. Nutzen Sie `"\t"` für die Spaltenabstände.

Erwarteter Dateiinhalt (Auszug):

```
1 t = 0 s      s = 0.0 m
2 t = 1 s      s = 4.905 m
3 t = 2 s      s = 19.62 m
4 ...
5 t = 15 s     s = 1103.625 m
```